

Docker

Engine, daemon, images, containers, volumes, compose, swarm, hub, desktop

Соборова Виктория Викторовна, старший преподаватель каф. ИУ9

Docker

Docker — программное обеспечение с открытым исходным кодом, применяемое для разработки, тестирования, доставки и запуска веб-приложений в средах с поддержкой контейнеризации. Он нужен для более эффективного использования системы и ресурсов, быстрого развертывания готовых программных продуктов, а также для их масштабирования и переноса в другие среды с гарантированным сохранением стабильной работы.



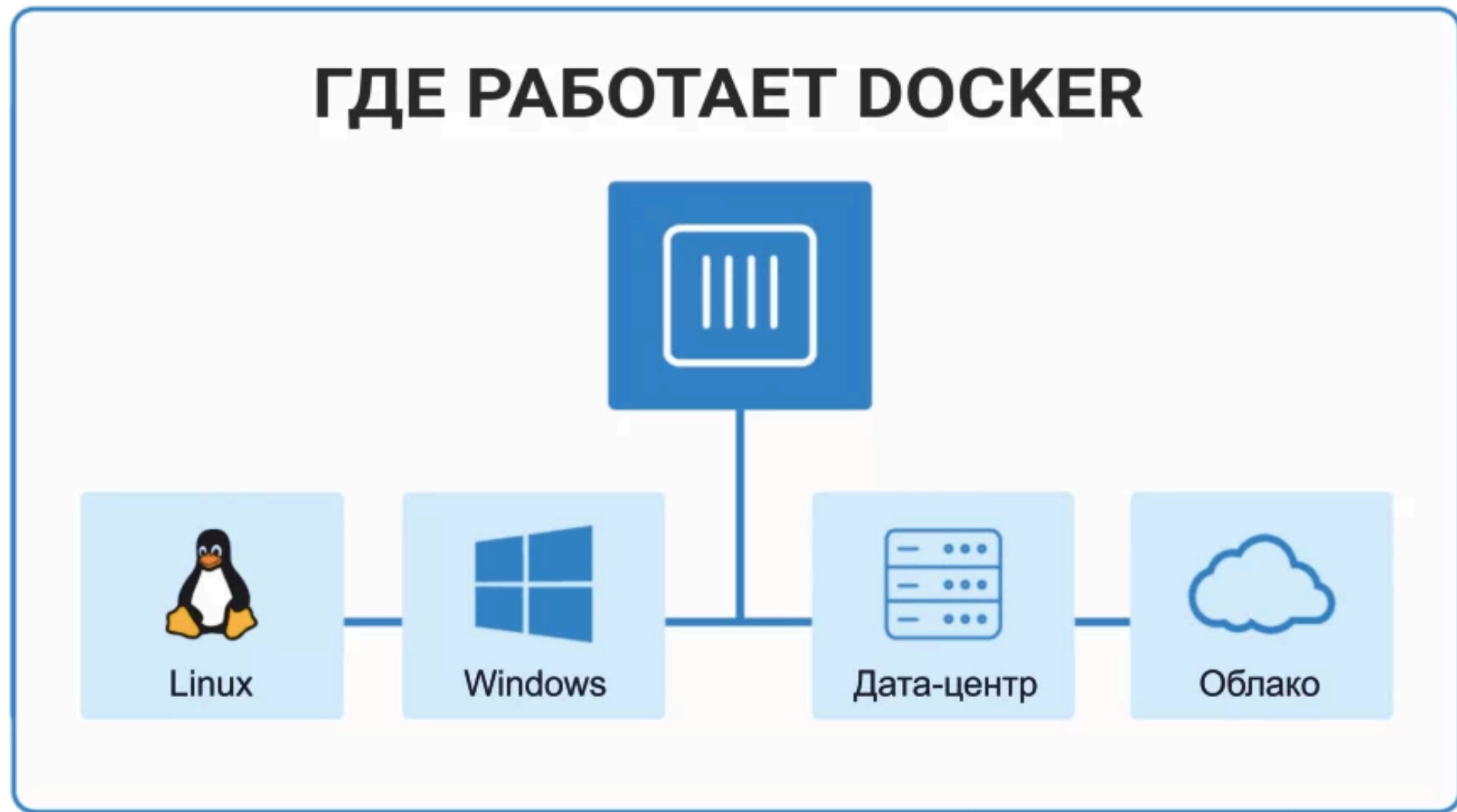
Docker

Разработка Docker была начата в 2008 году, а в 2013 году он был опубликован как свободно распространяемое ПО под лицензией Apache 2.0. В 2017 году была выпущена коммерческая версия Docker с расширенными возможностями.

Docker работает в Linux, ядро которых поддерживает cgroups, а также изоляцию пространства имен. Для инсталляции и использования на платформах, отличных от Linux, существуют специальные утилиты Kitematic или Docker Machine.

Основной принцип работы Docker — контейнеризация приложений. Этот тип виртуализации позволяет упаковывать программное обеспечение по изолированным средам — контейнерам.

Каждый из этих виртуальных блоков содержит все нужные элементы для работы приложения. Это дает возможность одновременного запуска большого количества контейнеров на одном хосте.



Docker-контейнеры работают в разных средах: локальном центре обработки информации, облаке, персональных компьютерах и т. д.

Docker- Как работает

Работа Docker основана на принципах клиент-серверной архитектуры, которая основана на взаимодействии клиента с веб-сервером (хостом). Первый отправляет запросы на получение данных, а второй их предоставляет.

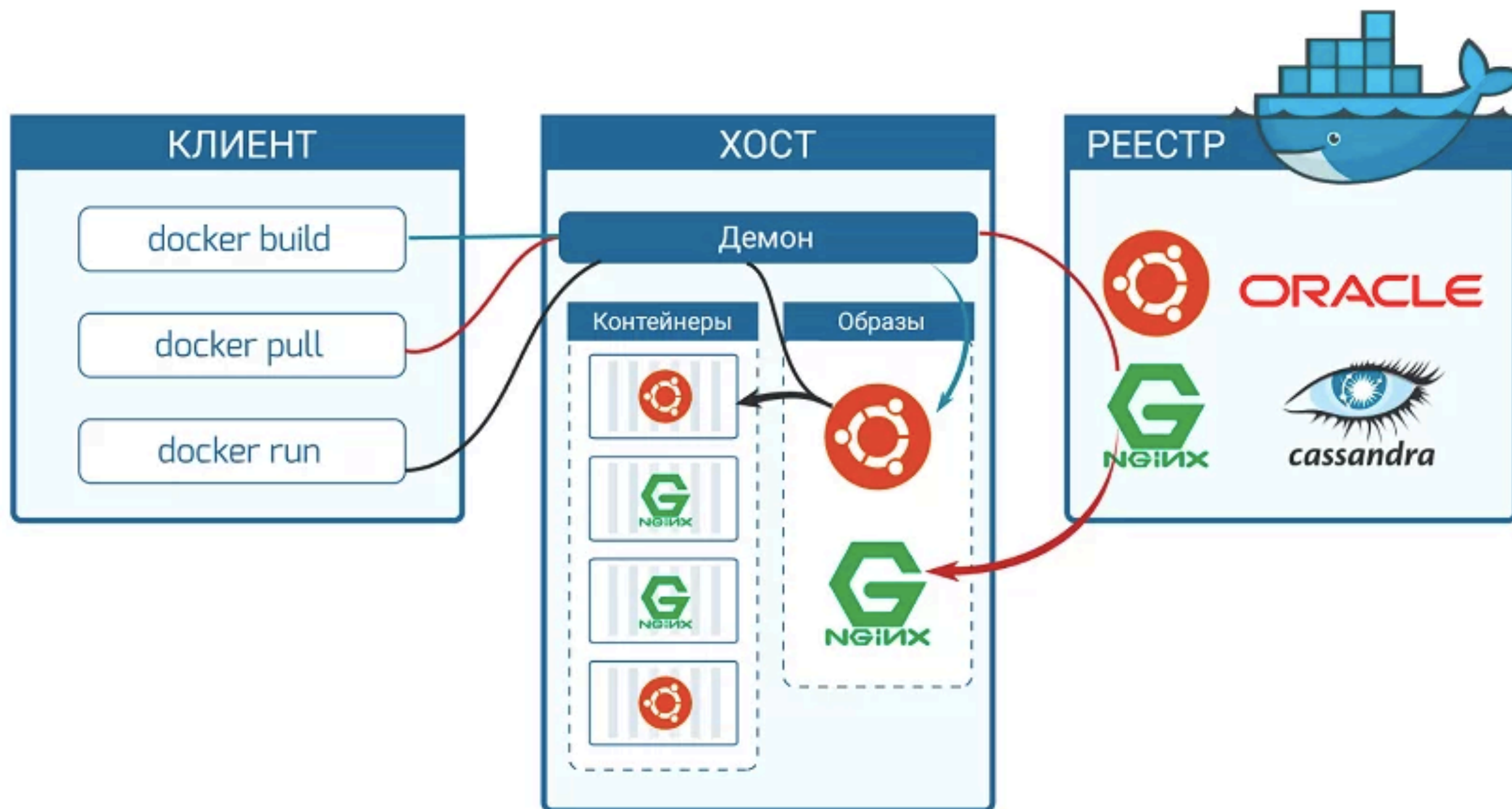
Схема работы

1. Пользователь отдает команду с помощью клиентского интерфейса Docker-демону, развернутому на Docker-хосте. Например, скачать готовый образ из реестра (хранилища Docker-образов) с помощью команды `docker pull`. Взаимодействие между клиентом и демоном обеспечивает REST API. Демон может использовать публичный (Docker Hub) или частный реестры.
2. Исходя из команды, заданной клиентом, демон выполняет различные операции с образами на основе инструкций, прописанных в файле Dockerfile. Например, производит их автоматическую сборку с помощью команды `docker build`.
3. Работа образа в контейнере. Например, запуск `docker-image`, посредством команды `docker run` или удаление контейнера через команду `docker kill`.

Docker - Преимущества

- 1. Минимальное потребление ресурсов** — контейнеры не виртуализируют всю операционную систему (ОС), а используют ядро хоста и изолируют программу на уровне процесса. Последний потребляет намного меньше ресурсов локального компьютера, чем виртуальная машина.
- 2. Скоростное развертывание** — вспомогательные компоненты можно не устанавливать, а использовать уже готовые docker-образы. Например, не имеет смысла постоянно устанавливать и настраивать Linux Ubuntu. Достаточно 1 раз ее установить, создать образ и постоянно использовать, лишь обновляя версию при необходимости.
- 3. Удобное скрывание процессов** — для каждого контейнера можно использовать разные методы обработки данных, скрывая фоновые процессы.
- 4. Работа с небезопасным кодом** — технология изоляции контейнеров позволяет запускать любой код без вреда для ОС.
- 5. Простое масштабирование** — любой проект можно расширить, внедрив новые контейнеры.
- 6. Удобный запуск** — приложение, находящееся внутри контейнера, можно запустить на любом docker-хосте.
- 7. Оптимизация файловой системы** — образ состоит из слоев, которые позволяют очень эффективно использовать файловую систему.

КОМПОНЕНТЫ DOCKER

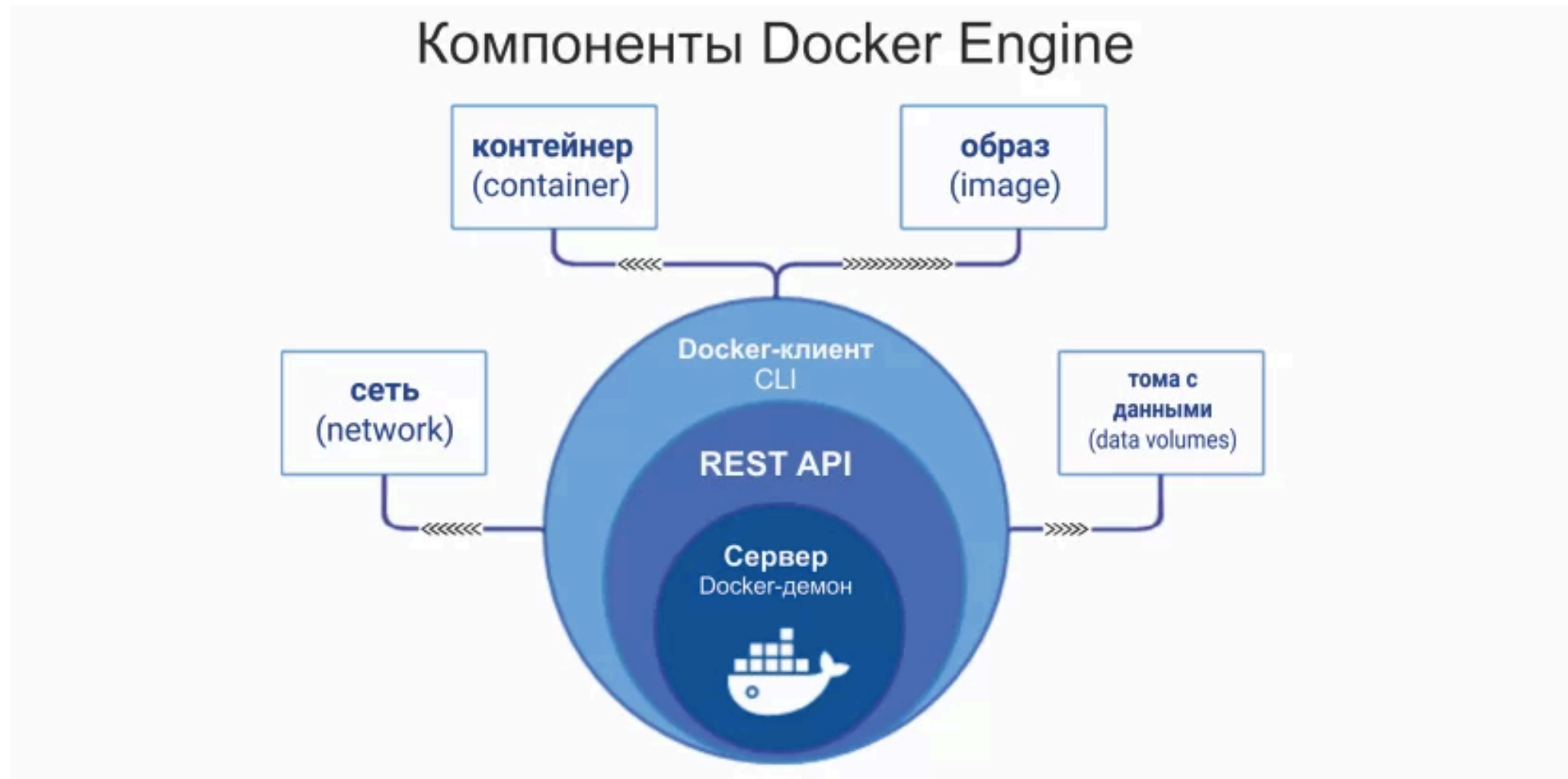


Docker - Компоненты

- 1. Docker-демон** (Docker-daemon) — сервер контейнеров, входящий в состав программных средств Docker. Демон управляет Docker-объектами (сети, хранилища, образы и контейнеры). Демон также может связываться с другими демонами для управления сервисами Docker.
- 2. Docker-клиент** (Docker-client / CLI) — интерфейс взаимодействия пользователя с Docker-демоном. Клиент и Демон — важнейшие компоненты «движка» Докера (Docker Engine). Клиент Docker может взаимодействовать с несколькими демонами.
- 3. Docker-образ** (Docker-image) — файл, включающий зависимости, сведения, конфигурацию для дальнейшего развертывания и инициализации контейнера.
- 4. Docker-файл** (Dockerfile) — описание правил по сборке образа, в котором первая строка указывает на базовый образ. Последующие команды выполняют копирование файлов и установку программ для создания определенной среды для разработки.
- 5. Docker-контейнер** (Docker-container) — это легкий, автономный исполняемый пакет программного обеспечения, который включает в себя все необходимое для запуска приложения: код, среду выполнения, системные инструменты, системные библиотеки и настройки.
- 6. Том** (Volume) — эмуляция файловой системы для осуществления операций чтения и записи. Она создается автоматически с контейнером, поскольку некоторые приложения осуществляют сохранение данных.
- 7. Реестр** (Docker-registry) — зарезервированный сервер, используемый для хранения docker-образов.
Примеры реестров:
 - **Центр Docker** — реестр, используемый для загрузки docker-image. Он обеспечивает их размещение и интеграцию с GitHub и Bitbucket.
 - **Контейнеры Azure** — предназначен для работы с образами и их компонентами в директории Azure (Azure Active Directory).
 - **Доверенный реестр Docker** или DTR — служба docker-реестра для инсталляции на локальном компьютере или сети компании.
- 8. Docker-хаб** (Docker-hub) или хранилище данных — репозиторий, предназначенный для хранения образов с различным программным обеспечением. Наличие готовых элементов влияет на скорость разработки.
- 9. Docker-хост** (Docker-host) — машинная среда для запуска контейнеров с программным обеспечением.
- 10. Docker-сети** (Docker-networks) — применяются для организации сетевого интерфейса между приложениями, развернутыми в контейнерах.

Docker Engine

Docker Engine («Движок» Docker) — ядро механизма Докера. «Движок» отвечает за функционирование и обеспечение связи между основными Docker-объектами (реестром, образами и контейнерами).



1. **Сервер** выполняет инициализацию демона (фоновой программы), который применяется для управления и модификации контейнеров, образов и томов.
2. **REST API** — механизм, отвечающий за организацию взаимодействия Докер-клиента и Докер-демона.
3. **Клиент** — позволяет пользователю взаимодействовать с сервером при помощи команд, набираемых в интерфейсе (CLI).

Docker Images

Docker-image — шаблон только для чтения (read-only) с набором некоторых инструкций, предназначенных для создания контейнера. Он состоит из слоев, которые Docker комбинирует в один образ при помощи вспомогательной файловой системы UnionFS. Так решается проблема нерационального использования дисковой памяти. Параметры образа определяются в Docker-file.

Для многократного применения Docker-image следует пользоваться реестром образов или Докер-реестром (Docker-registry), позволяющим зачислять готовые образы с внешнего репозитория сервиса и хранить их в реестре Докер-хоста. Рекомендуемый вариант — официальный реестр компании Docker Trusted Registry (DTR).

Если требуется файл, то скачиваться будут только нужные слои. Например, разработчик решил доработать программное обеспечение и модифицировать образ, изменив несколько файлов. После загрузки на сервер будут отправлены слои, содержащие только модифицированные данные.

Docker Containers

Каждый контейнер строится на основе Docker-образов. Контейнеры запускаются напрямую из ядра операционной системы Linux. Благодаря этому, они потребляют гораздо меньше ресурсов, чем при аппаратной виртуализации.

Изоляция рабочей среды осуществляется при помощи технологии namespaces. Для каждого изолированного пространства (контейнера) создается уникальное пространство имен, которое и обеспечивает к нему доступ. Любой процесс, выполняемый внутри контейнера, ограничивается namespaces.

Что происходит при запуске контейнера

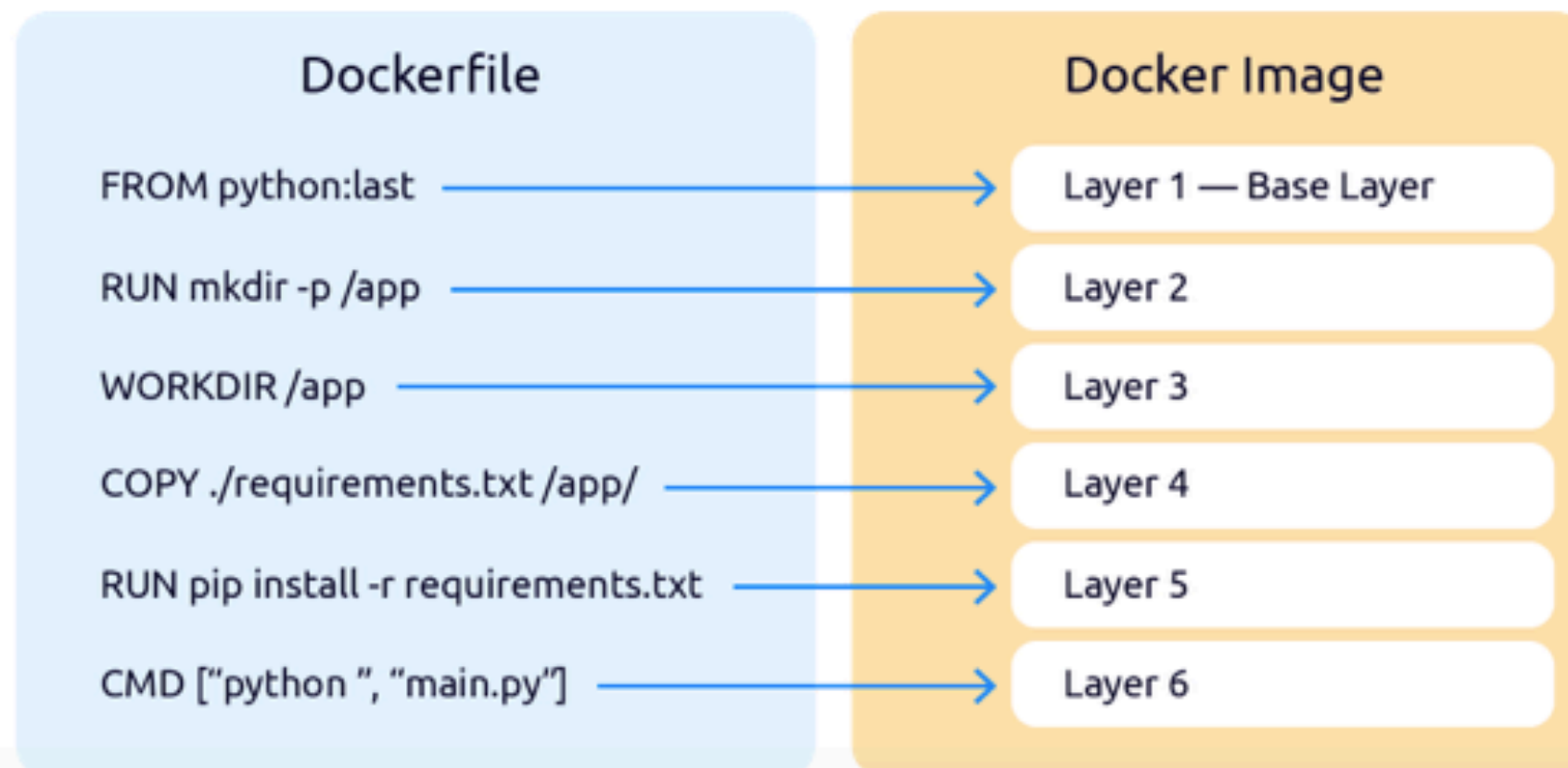
1. Происходит запуск образа (Docker-image). Docker Engine проверяет существование образа. Если образ уже существует локально, Docker использует его для нового контейнера. При его отсутствии выполняется скачивание с Docker Hub.
2. Создание контейнера из образа.
3. Разметка файловой системы и добавление слоя для записи.
4. Создание сетевого интерфейса.
5. Поиск и присвоение IP-адреса.
6. Запуск указанного процесса.
7. Захват ввода/вывода приложения.

Docker Containers



Dockerfile — это конфигурационный файл с инструкциями по созданию Docker-образов. Почти каждая команда инструкции создаёт новый слой в образе. Это нужно для дальнейшего использования уже готовых слоев.

Например, мы изменили последнюю инструкцию в Dockerfile и собрали образ заново. Docker не будет заново всё пересобирать. Он возьмет все предшествующие слои и переиспользует их. Вот как образно выглядит соотношение инструкций в Dockerfile со слоями в Docker-образе:



Docker Containers

1. FROM

Любой код или набор инструкций выполняется сверху вниз. Поэтому Dockerfile всегда начинается с открывающей инструкции **FROM**, которая говорит демону Docker, какой образ для основы нужно взять. Если образа локально нет — он будет скачан с Docker hub.

2. RUN

Далее идет инструкция **RUN** с определенной командой. В нашем случае мы использовали команду **mkdir -p** для создания папки /app. Инструкций **RUN** может быть неограниченное количество, но стоит помнить, что каждая инструкция создает свой слой, поэтому хорошей практикой является запись цепочки команд через **&&**: **RUN comand_1 && comand_2 && comand_3**.

3. WORKDIR

Есть инструкции логически связанные между собой. Инструкция **WORKDIR** устанавливает активный рабочий каталог. Все последующие команды, такие как **COPY**, **RUN**, **CMD** и некоторые другие будут выполнены из рабочего каталога, установленного через **WORKDIR**.

4. COPY

С инструкцией **COPY** всё просто. Первым аргументом указывается папка для копирования, а вторым аргументом — папка в контейнере куда будут помещены файлы из копируемой директории. В нашем случае, из текущего каталога (".") — каталог, в котором находится пользователь, ".." — каталог выше относительно "."), в котором находится пользователь, был скопирован файл requirements.txt и помещен в папку /app в контейнере.

5. RUN

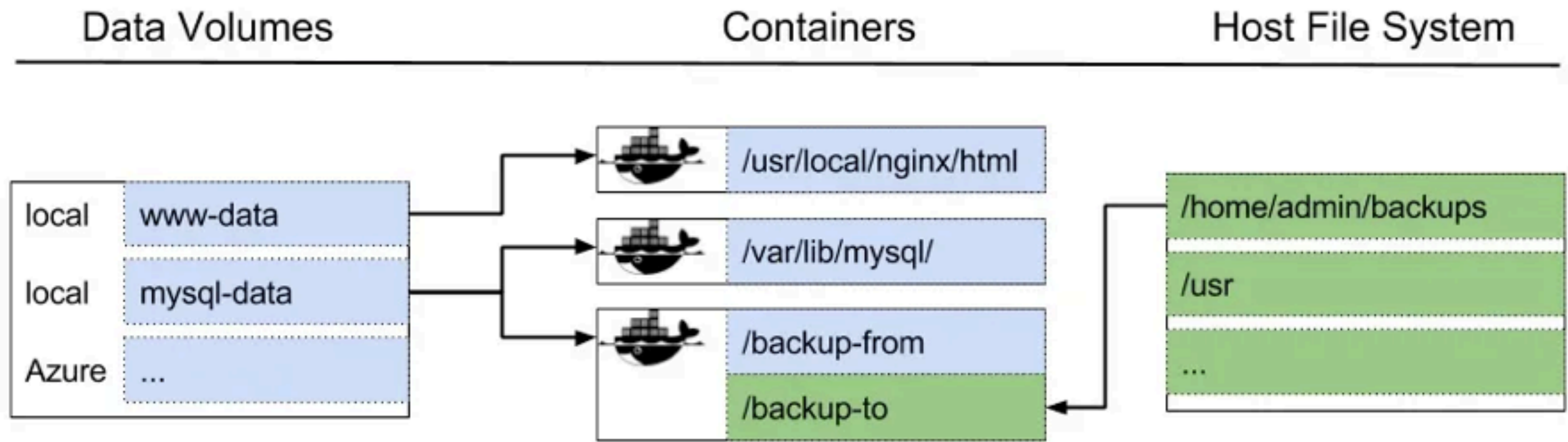
Инструкцию **RUN** мы рассмотрели ранее. Тут лишь хотим обратить ваше внимание на её поведение в сочетании с инструкцией **WORKDIR**. Ранее инструкция **COPY** перенесла файл requirements.txt в контейнер. Кстати, в качестве финального пути мы могли указать ".", так как инструкция **WORKDIR** установила в качестве рабочей директории контейнера папку /app. И теперь команда **RUN** будет выполнена именно из директории /app.

6. CMD

Финальной инструкцией в любом Dockerfile является **CMD** или **ENTRYPOINT**. В отличие от других инструкций **CMD** может быть только одна и она может быть переопределена при старте контейнера командой **docker run**. Инструкция **CMD** наследует условия установленные инструкцией **WORKDIR**.

Docker Volume

Docker volume — это папка хоста, примонтированная к файловой системе контейнера. Так как технически она больше не принадлежит контейнеру, то последний можно смело удалять, пересоздавать заново, снова прикручивать к нему хостовые папки, и ничего с данными внутри не случится.



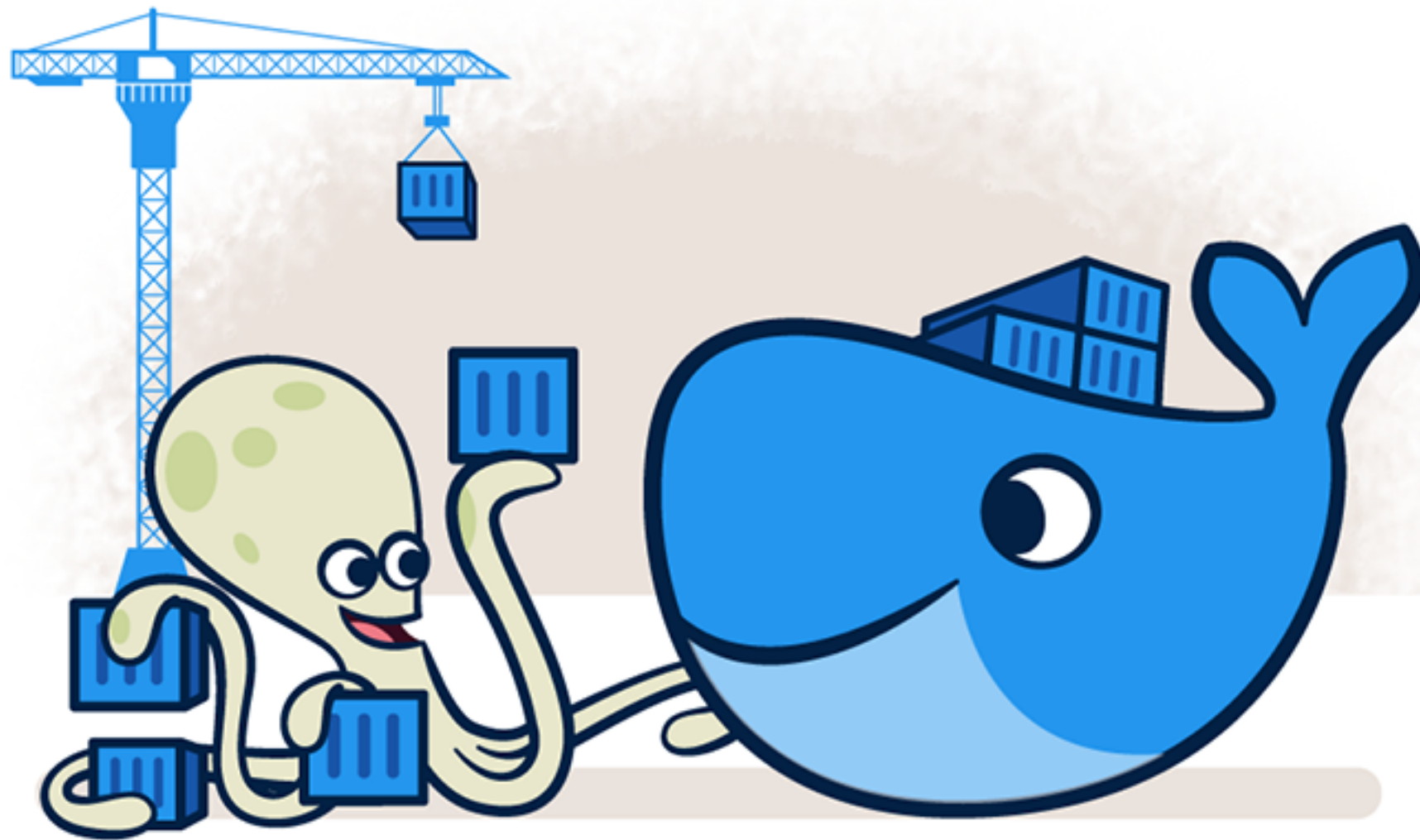
Docker Compose

Для управления несколькими контейнерами, из которых состоит проект, используют пакетный менеджер — [Docker Compose](#).

Он применяется не во всех случаях. Если проект является простым приложением, не требующим использования сторонних сервисов, то для его развертывания можно ограничиться только Docker. Docker Compose рекомендуется использовать при проектировании сложных программных продуктов, включающих в себя множество процессов и сервисов.

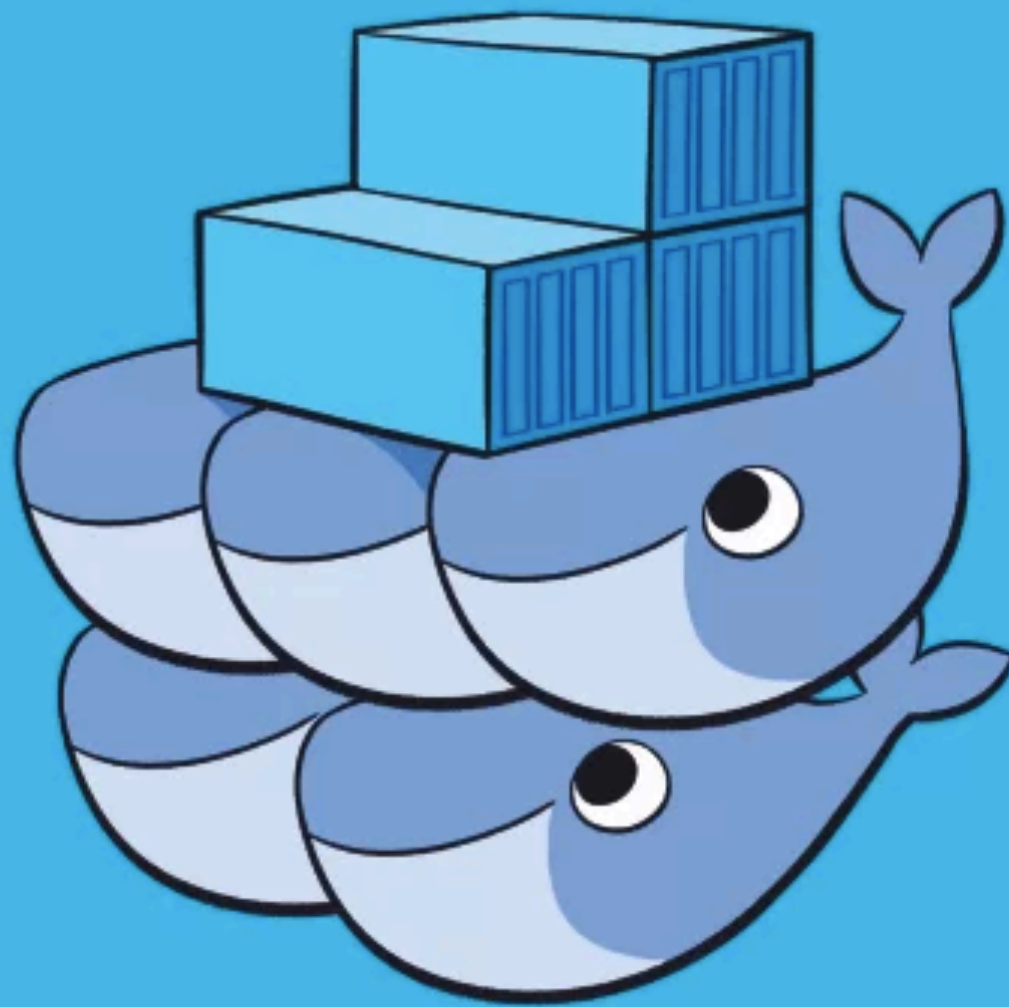


Docker Compose



Docker Swarm

При преобразовании хостов в кластер нужно воспользоваться утилитой кластеризации **Docker Swarm** («Docker в режиме роя»). Хост, находящийся в его составе, называется «узлом» (node), который бывает управляющим или рабочим. Один кластер содержит только один управляющий «узел».



Docker Hub — это общедоступный Docker registry, то есть хранилище всех доступных Docker-образов. При необходимости можно разворачивать свои приватные Docker registry, размещать собственные реестры Docker и использовать их для извлечения образов.

Integration Tests



Docker Hub



Deployment Platforms



Google Compute Engine

Revision Control



boot2docker

Docker Desktop - это собственное настольное приложение, разработанное Docker для пользователей Windows и MAC. Это самый простой способ запуска, сборки, отладки и тестирования приложений Dockerized.

Docker Desktop предлагает важные и наиболее полезные функции, такие как быстрые циклы редактирования, уведомления об изменениях файлов, встроенная поддержка корпоративной сети и гибкость для работы с собственным выбором прокси и VPN.

Docker Desktop состоит из инструментов для разработчика, приложения Docker, Kubernetes и синхронизации версий. Он позволяет вам создавать сертифицированные образы и шаблоны языков и инструментов.

Docker Command

[Установить Homebrew]

//адрес официального сайта на русском языке

https://brew.sh/index_ru

//команда для терминала

/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

//обновление

brew upgrade

Docker Command

[Установка Docker]

//установка

brew install docker

//установка образов images

docker run php

docker run phpmyadmin

docker run mysql

docker run httpd

//проверка какие образы установлены

docker images

//проверка созданных контейнеров и запущенных в данный момент

docker ps

//проверка всех созданных контейнеров

docker ps -a

//запуск контейнера httpd

docker run -d -p 80:80 httpd

//узнать id контейнера от имени пользователя root@<container_id>

docker run -it <IMAGE> /bin/bash

//запустить ранее созданный контейнер

docker start <CONTAINER_ID>

//остановить контейнер

docker stop <CONTAINER_ID>

//удалить контейнер

docker rm <CONTAINER_ID>

//создать объем докера для постоянных данных

docker volume create apache-data

//проявить постоянный каталог данных

docker volume inspect apache-data

//сборка проекта

docker-compose up --build

Docker Command

[Окружение Docker]

//комплект программного обеспечения, окружение

//стек LAMP (Linux Apache MySQL PHP)

//стек WAMP (Windows Apache MySQL PHP)

//стек MAMP (Mac Apache MySQL PHP)

//стек SAMP (Solaris Apache MySQL PHP)

//создание директории проекта

mkdir -p /Users/victoriasoborova/Sites/[project_name]

//открыть директорию

cd /Users/victoriasoborova/Sites/[project_name]

Docker Command

[MySQL Server в Docker]

//установка образа mysql-server последней версии

docker pull mysql/mysql-server:latest

//разворачиваем контейнер из образа mysql-server

docker run --name=mysql.server -d mysql/mysql-server:latest

//убедимся, что контейнер создан и запущен

docker ps

//во время развертывания был создан случайный пароль

//проверка логина и пароля

docker logs mysql.server

GENERATED ROOT PASSWORD: 4/3z3PBTAiSk+8?uiA@339l*+K2G=zo%

//заходим в контейнеризированный сервер mysql

docker exec -it mysql.server mysql -uroot -p

// -it -- это означает интегрировать терминал, значит консоль будет внутри кнтейнера mysql.server

//меняем пароль пользователя root

alter user 'root'@'localhost' identified by 'rootpass';

Docker Command

[Создание Базы данных]

//просмотр всех баз данных

show databases;

//создание базы данных travel

create database travel;

//просмотр всех таблиц в базе данных travel

show tables in travel;

//удаление базы данных

drop database travel;

//выбрать базу данных

use travel

//просмотр таблиц в выбранной базе данных

show tables;

Docker Command

/*СОЗДАНИЕ ТАБЛИЦ*/

```
CREATE TABLE IF NOT EXISTS `Hotel` (  
  
  `Id_hotel` INT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
  
  `Name_hotel` varCHAR(60) NOT NULL,  
  
  `Class_hotel` INT NOT NULL);  
  
CREATE TABLE `Turist` (  
  
  `Id_turist` INT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
  
  `SecondName` varCHAR(60) NOT NULL,  
  
  `FirstName` varCHAR(60) NOT NULL,  
  
  `Patronym` varCHAR(60),  
  
  `SeriaPassport` INT(4) NOT NULL,  
  
  `NumberPassport` INT(6) NOT NULL);  
  
CREATE TABLE `Turoperator`(  
  
  `Id_turoperator` INT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
  
  `Name_turoperator` varCHAR(60) NOT NULL);  
  
CREATE TABLE `Putevka`(  
  
  `Id_putevka` INT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
  
  `Number_putevka` INT NOT NULL,  
  
  `DateOtpravki` DATE NOT NULL,  
  
  `DateVozvr` DATE NOT NULL,  
  
  `Hotel` INT NOT NULL REFERENCES Hotel(Id_hotel),  
  
  `Turoperator` INT NOT NULL REFERENCES Turoperator(Id_turoperator),  
  
  `Turist` INT NOT NULL REFERENCES Turist(Id_turist));
```

/*ВВОД ДАННЫХ*/

```
INSERT INTO `Hotel` VALUES  
  
(1, "Rosa Resort Hotel", 4),  
  
(2, "Joy World Palace", 5),  
  
(3, "My Dream Beach", 3),  
  
(4, "Golden Beach Hotel ", 4),  
  
(5, "Joy Kirish Resort", 5);  
  
INSERT INTO `Turist` VALUES  
  
(1, "Antonov", "Anton", "Aleksandrovich", 1234, 567456),  
  
(2, "Smirnov", "Aleksandr", "Sergeevich", 1234, 567346),  
  
(3, "Ivanov", "Ivan", "Alekseevich", 2351, 346234),  
  
(4, "Sidorova", "Anastasia", "Andreevna", 9673, 956784),  
  
(5, "Smirnova", "Lisa", "Sergeevna", 2342,623495);  
  
INSERT INTO `Turoperator` VALUES  
  
(1, "TEZ tour"),  
  
(2, "Anex tour"),  
  
(3, "Coral travel"),  
  
(4, "TUI");  
  
INSERT INTO `Putevka` VALUES  
  
(1, 111, '2018-05-03', '2018-05-20', 3, 2, 5),  
  
(2, 111, '2018-05-03', '2018-05-20', 3, 2, 2),  
  
(3, 113, '2019-08-01', '2019-08-15', 1, 1, 1),  
  
(4, 113, '2019-08-01', '2019-08-15', 1, 1, 4);
```

Docker Command

```
//просмотреть структуру таблицы Hotel
describe Hotel;
mysql> describe Hotel;
```

Field	Type	Null	Key	Default	Extra
Id_hotel	int	NO	PRI	NULL	auto_increment
Name	varchar(100)	NO		NULL	
Class	int	NO		NULL	

3 rows in set (0,05 sec)

```
mysql> select * from Hotel;
```

Id_hotel	Name	Class
1	Rosa Resort Hotel	4
2	Joy World Palace	5
3	My Dream Beach	3
4	Golden Beach Hotel	4
5	Joy Kirish Resort	5

5 rows in set (0,00 sec)

Docker Command

//просмотр текущих процессов

show processlist;

//бэкап базы данных в файл travel-dump.sql, в выбранную директорию

//пароль лучше не указывать, а вводить в скрытом фиде

docker exec mysql.server /usr/bin/mysqldump -uroot -prootpass travel > /Users/victoriasoborova/Sites/travel-dump.sql

//развернуть бэкап из файла

cat /Users/victoriasoborova/Sites/travel-dump2.sql | docker exec -i mysql.server /usr/bin/mysql -uroot -prootpass travel2

//просмотр всех триггеров в базе данных travel

show triggers from travel;

//просмотр триггеров в базе данных у таблицы Hotel

show triggers in travel LIKE'Hotel';

mysql> use travel

Reading table information for completion of table and column names

You can turn off this feature to get a quicker startup with -A

Docker Command

//создание директории проекта

mkdir -p /Users/victoriasoborova/Sites/site

//создание файлов в brackets

sites

- index.php

- docker-compose.yml

//написание docker-compose.yml

version: '3.8'

services:

php-apache-environment:

container_name: php-apache

image: php:8.0-apache

volumes:

- ./site:/var/www/html

ports:

- 9000:80

//написание index.php

<?php

echo "hello! its my website with docker!";

?>

//открытие директории

cd /Users/victoriasoborova/Sites/sites

//запуск сборки

docker-compose up

//открытие site

Localhost:9000